

ProjectAAI628_model_5

December 20, 2025

```
[1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import random
import seaborn as sns

import tensorflow as tf
from tensorflow.keras.optimizers import Adam
from sklearn.metrics import accuracy_score, precision_score, recall_score, \
    precision_recall_curve
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers, losses, Model
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from sklearn.metrics import f1_score
from tensorflow.keras.models import Model

np.random.seed(42)
tf.random.set_seed(42)
random.seed(42)
```

```
[2]: # Download the dataset
dataframe = pd.read_csv('http://storage.googleapis.com/download.tensorflow.org/
    data/ecg.csv', header=None)
raw_data = dataframe.values
dataframe.head()
```

```
[2]:
```

	0	1	2	3	4	5	6	\
0	-0.112522	-2.827204	-3.773897	-4.349751	-4.376041	-3.474986	-2.181408	
1	-1.100878	-3.996840	-4.285843	-4.506579	-4.022377	-3.234368	-1.566126	
2	-0.567088	-2.593450	-3.874230	-4.584095	-4.187449	-3.151462	-1.742940	
3	0.490473	-1.914407	-3.616364	-4.318823	-4.268016	-3.881110	-2.993280	
4	0.800232	-0.874252	-2.384761	-3.973292	-4.338224	-3.802422	-2.534510	
	7	8	9	...	131	132	133	134 \
0	-1.818286	-1.250522	-0.477492	...	0.792168	0.933541	0.796958	0.578621
1	-0.992258	-0.754680	0.042321	...	0.538356	0.656881	0.787490	0.724046

```

2 -1.490659 -1.183580 -0.394229 ... 0.886073 0.531452 0.311377 -0.021919
3 -1.671131 -1.333884 -0.965629 ... 0.350816 0.499111 0.600345 0.842069
4 -1.783423 -1.594450 -0.753199 ... 1.148884 0.958434 1.059025 1.371682

```

```

          135          136          137          138          139  140
0  0.257740  0.228077  0.123431  0.925286  0.193137  1.0
1  0.555784  0.476333  0.773820  1.119621 -1.436250  1.0
2 -0.713683 -0.532197  0.321097  0.904227 -0.421797  1.0
3  0.952074  0.990133  1.086798  1.403011 -0.383564  1.0
4  1.277392  0.960304  0.971020  1.614392  1.421456  1.0

```

[5 rows x 141 columns]

[]:

```

[3]: # The last element contains the labels
labels = 1 - raw_data[:, -1]

# The other data points are the electrocardiogram data
data = raw_data[:, 0:-1]

```

```

[4]: train_data, test_data, train_labels, test_labels = train_test_split(
      data, labels, test_size=0.2, random_state=42, stratify=labels
    )

min_val = tf.reduce_min(train_data)
max_val = tf.reduce_max(train_data)

train_data = (train_data - min_val) / (max_val - min_val)
test_data = (test_data - min_val) / (max_val - min_val)

train_data = tf.cast(train_data, tf.float32)
test_data = tf.cast(test_data, tf.float32)

```

```

[5]: train_labels = train_labels.astype(bool)
test_labels = test_labels.astype(bool)

```

```

[6]: normal_train_data = train_data[~train_labels]
normal_test_data = test_data[~test_labels]

print(normal_test_data.shape)
print(normal_train_data.shape)

```

(584, 140)

(2335, 140)

```

[7]: anomalous_train_data = train_data[train_labels]
anomalous_test_data = test_data[test_labels]

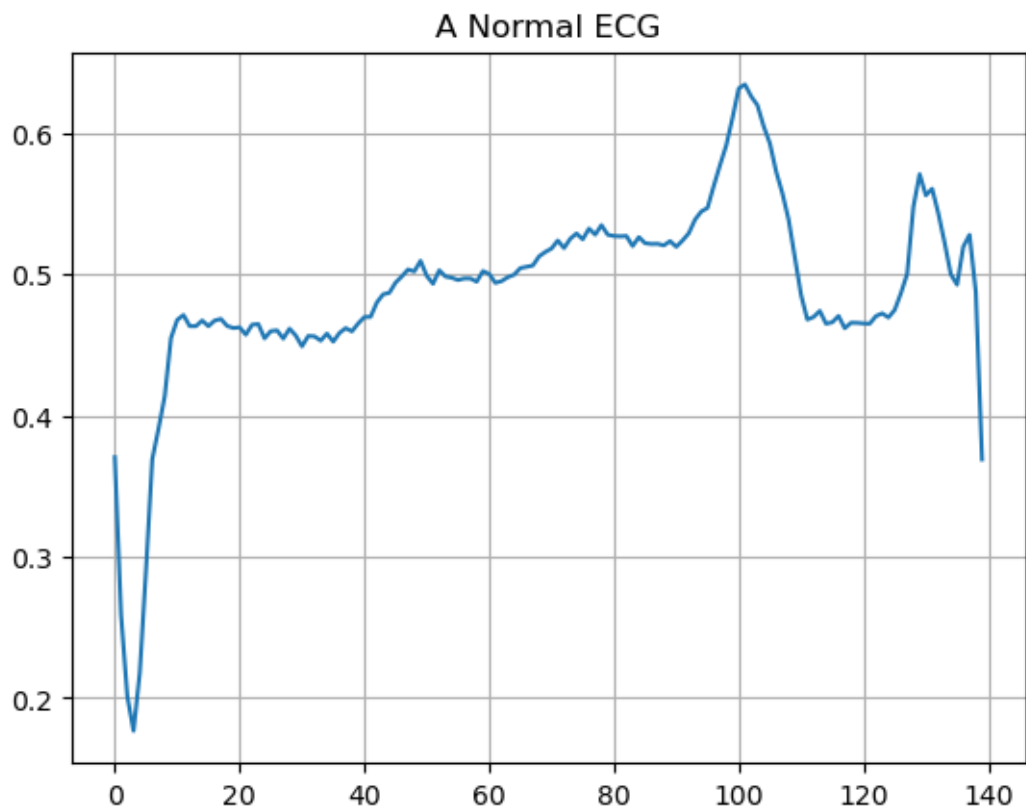
```

```
[8]: print(anomalous_test_data.shape)
      print(anomalous_train_data.shape)
```

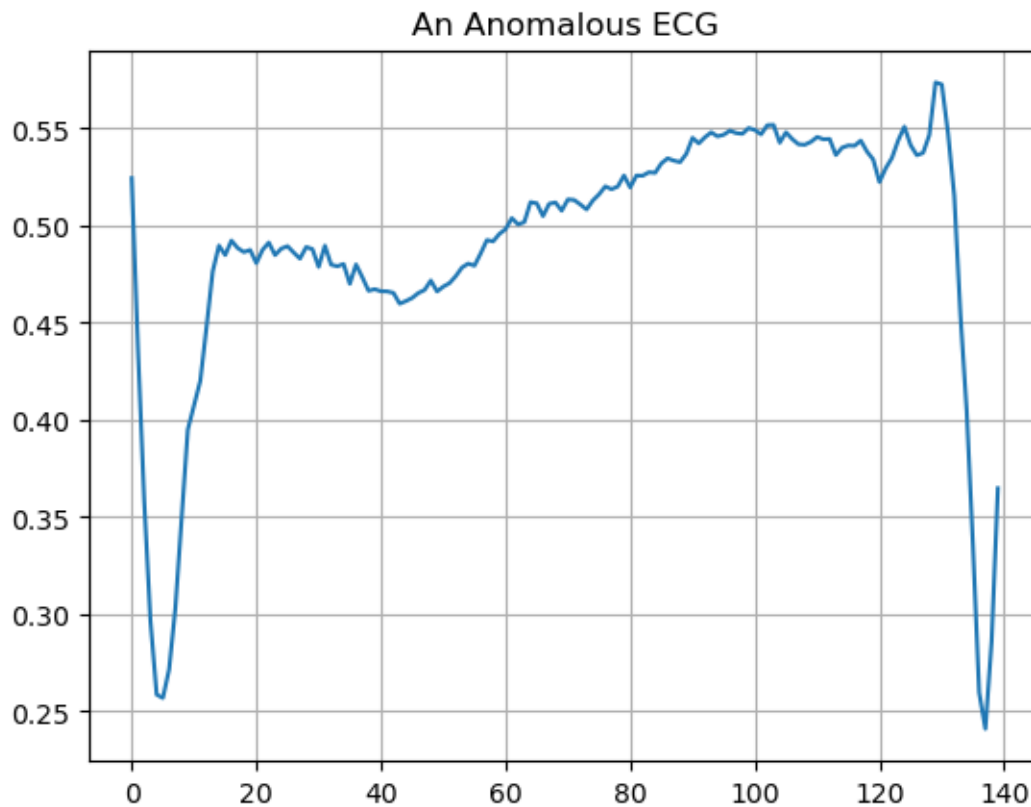
```
(416, 140)
(1663, 140)
```

```
[9]: # print("Train normals:", X_train_normal.shape)
      # print("Train anomalies:", np.sum(y_train == 1))
      # print("Test normals:", np.sum(y_test == 0))
      # print("Test anomalies:", np.sum(y_test == 1))
```

```
[10]: plt.grid()
      plt.plot(np.arange(140), normal_train_data[0])
      plt.title("A Normal ECG")
      plt.show()
```



```
[11]: plt.grid()
      plt.plot(np.arange(140), anomalous_train_data[0])
      plt.title("An Anomalous ECG")
      plt.show()
```



```
[12]: # plt.plot(normal_train_data[0], label="Normal")
# plt.plot(anomalous_train_data[0], label="Anomalous")
# plt.legend()
# plt.title("Normal vs Anomalous ECG")
# plt.show()
```

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```

```
[13]: #Model 5

class CNNAnomalyDetector(tf.keras.Model):
    def __init__(self):
        super().__init__()

        # Encoder
        self.encoder = tf.keras.Sequential([
            layers.Conv1D(32, 5, padding="same"),
```

```

        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.1),
        layers.MaxPooling1D(2, padding="same"),
        layers.Dropout(0.2),

        layers.Conv1D(64, 5, padding="same"),
        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.1),
        layers.MaxPooling1D(2, padding="same"),
        layers.Dropout(0.2),

        layers.Conv1D(128, 3, padding="same"),
        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.1),
        layers.MaxPooling1D(2, padding="same"),
        layers.Dropout(0.5)
    ])

    # Decoder
    self.decoder = tf.keras.Sequential([
        layers.Conv1D(128, 3, padding="same"),
        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.1),
        layers.UpSampling1D(2),
        layers.Dropout(0.5),

        layers.Conv1D(64, 5, padding="same"),
        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.1),
        layers.UpSampling1D(2),
        layers.Dropout(0.2),

        layers.Conv1D(32, 5, padding="same"),
        layers.BatchNormalization(),
        layers.LeakyReLU(alpha=0.1),
        layers.UpSampling1D(2),
        layers.Dropout(0.1),

        # IMPORTANT: linear output for ECG reconstruction
        layers.Conv1D(1, 3, activation="linear", padding="same"),
        layers.Cropping1D((2, 2)) # 144 → 140
    ])

    def call(self, x):
        return self.decoder(self.encoder(x))

```

```

autoencoder = CNNAomalyDetector()
autoencoder.compile(
    optimizer=tf.keras.optimizers.Nadam(learning_rate=1e-3),
    loss="mae"
)
_ = autoencoder(tf.zeros((1, 140, 1)))
autoencoder.summary()

```

C:\Users\ultra\anaconda3\Lib\site-packages\keras\src\layers\activations\leaky_relu.py:41: UserWarning: Argument `alpha` is deprecated. Use `negative\_slope` instead.

warnings.warn(

Model: "cnn_anomaly_detector"

Layer (type)	Output Shape	
Param #		
sequential (Sequential)	(1, 18, 128)	
36,096		
sequential_1 (Sequential)	(1, 140, 1)	
101,569		

Total params: 137,665 (537.75 KB)

Trainable params: 136,769 (534.25 KB)

Non-trainable params: 896 (3.50 KB)

```

[14]: callbacks = tf.keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=5,
    restore_best_weights=True
)

```

[]:

```

[15]: history = autoencoder.fit(normal_train_data, normal_train_data,
    epochs=100,
    batch_size=128,
    validation_data=(normal_test_data, normal_test_data),
    shuffle=True,

```

```
callbacks=callbacks
)
```

```
Epoch 1/100
19/19          3s 24ms/step -
loss: 0.1194 - val_loss: 0.0392
Epoch 2/100
19/19          0s 12ms/step -
loss: 0.0455 - val_loss: 0.0413
Epoch 3/100
19/19          0s 12ms/step -
loss: 0.0313 - val_loss: 0.0235
Epoch 4/100
19/19          0s 12ms/step -
loss: 0.0247 - val_loss: 0.0268
Epoch 5/100
19/19          0s 12ms/step -
loss: 0.0217 - val_loss: 0.0194
Epoch 6/100
19/19          0s 12ms/step -
loss: 0.0200 - val_loss: 0.0236
Epoch 7/100
19/19          0s 12ms/step -
loss: 0.0193 - val_loss: 0.0167
Epoch 8/100
19/19          0s 12ms/step -
loss: 0.0186 - val_loss: 0.0203
Epoch 9/100
19/19          0s 12ms/step -
loss: 0.0182 - val_loss: 0.0157
Epoch 10/100
19/19          0s 11ms/step -
loss: 0.0174 - val_loss: 0.0178
Epoch 11/100
19/19          0s 11ms/step -
loss: 0.0173 - val_loss: 0.0157
Epoch 12/100
19/19          0s 11ms/step -
loss: 0.0165 - val_loss: 0.0196
Epoch 13/100
19/19          0s 12ms/step -
loss: 0.0165 - val_loss: 0.0146
Epoch 14/100
19/19          0s 12ms/step -
loss: 0.0159 - val_loss: 0.0186
Epoch 15/100
19/19          0s 12ms/step -
loss: 0.0156 - val_loss: 0.0146
```

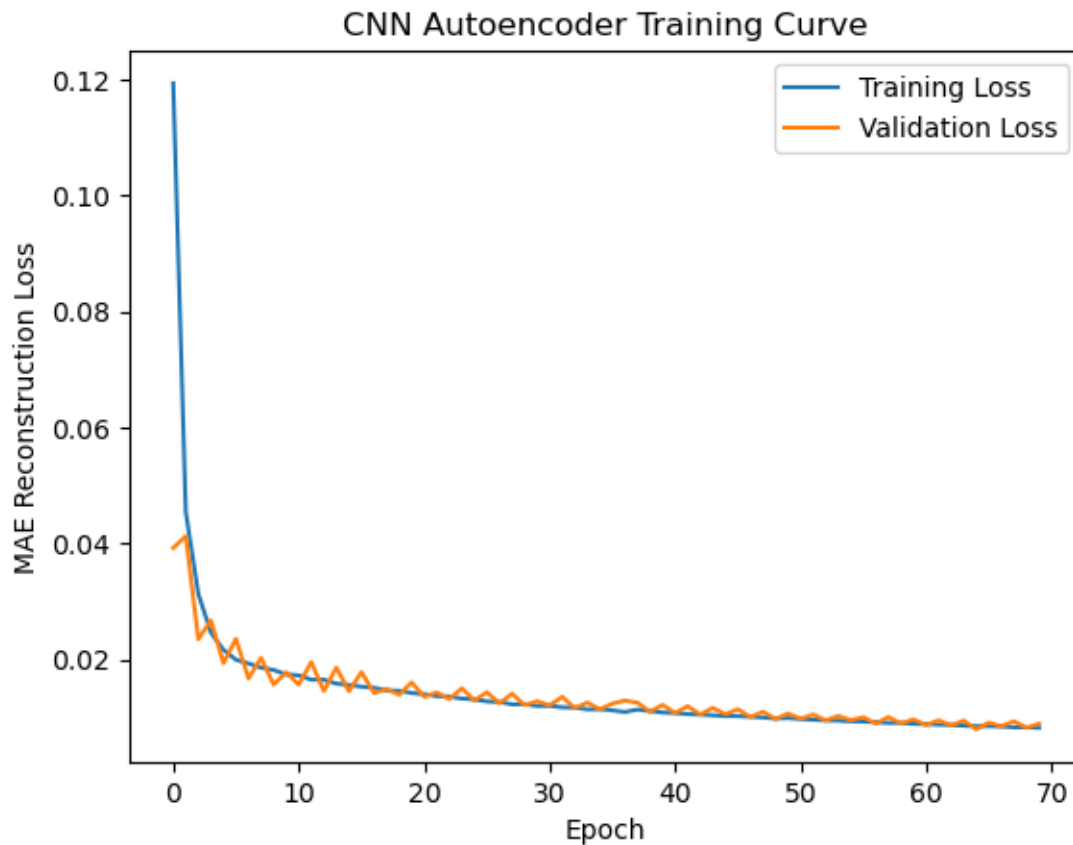
Epoch 16/100
19/19 0s 11ms/step -
loss: 0.0153 - val_loss: 0.0178
Epoch 17/100
19/19 0s 12ms/step -
loss: 0.0151 - val_loss: 0.0142
Epoch 18/100
19/19 0s 12ms/step -
loss: 0.0147 - val_loss: 0.0149
Epoch 19/100
19/19 0s 11ms/step -
loss: 0.0146 - val_loss: 0.0139
Epoch 20/100
19/19 0s 11ms/step -
loss: 0.0142 - val_loss: 0.0161
Epoch 21/100
19/19 0s 12ms/step -
loss: 0.0140 - val_loss: 0.0136
Epoch 22/100
19/19 0s 11ms/step -
loss: 0.0137 - val_loss: 0.0143
Epoch 23/100
19/19 0s 12ms/step -
loss: 0.0136 - val_loss: 0.0132
Epoch 24/100
19/19 0s 12ms/step -
loss: 0.0133 - val_loss: 0.0150
Epoch 25/100
19/19 0s 11ms/step -
loss: 0.0131 - val_loss: 0.0129
Epoch 26/100
19/19 0s 11ms/step -
loss: 0.0128 - val_loss: 0.0144
Epoch 27/100
19/19 0s 11ms/step -
loss: 0.0127 - val_loss: 0.0124
Epoch 28/100
19/19 0s 11ms/step -
loss: 0.0123 - val_loss: 0.0141
Epoch 29/100
19/19 0s 11ms/step -
loss: 0.0123 - val_loss: 0.0121
Epoch 30/100
19/19 0s 11ms/step -
loss: 0.0120 - val_loss: 0.0128
Epoch 31/100
19/19 0s 11ms/step -
loss: 0.0120 - val_loss: 0.0121

Epoch 32/100
19/19 0s 11ms/step -
loss: 0.0118 - val_loss: 0.0136
Epoch 33/100
19/19 0s 11ms/step -
loss: 0.0117 - val_loss: 0.0116
Epoch 34/100
19/19 0s 11ms/step -
loss: 0.0114 - val_loss: 0.0126
Epoch 35/100
19/19 0s 11ms/step -
loss: 0.0114 - val_loss: 0.0114
Epoch 36/100
19/19 0s 11ms/step -
loss: 0.0112 - val_loss: 0.0124
Epoch 37/100
19/19 0s 11ms/step -
loss: 0.0110 - val_loss: 0.0129
Epoch 38/100
19/19 0s 11ms/step -
loss: 0.0114 - val_loss: 0.0126
Epoch 39/100
19/19 0s 11ms/step -
loss: 0.0111 - val_loss: 0.0110
Epoch 40/100
19/19 0s 11ms/step -
loss: 0.0109 - val_loss: 0.0122
Epoch 41/100
19/19 0s 12ms/step -
loss: 0.0108 - val_loss: 0.0108
Epoch 42/100
19/19 0s 12ms/step -
loss: 0.0106 - val_loss: 0.0120
Epoch 43/100
19/19 0s 11ms/step -
loss: 0.0105 - val_loss: 0.0105
Epoch 44/100
19/19 0s 11ms/step -
loss: 0.0104 - val_loss: 0.0117
Epoch 45/100
19/19 0s 11ms/step -
loss: 0.0103 - val_loss: 0.0106
Epoch 46/100
19/19 0s 11ms/step -
loss: 0.0102 - val_loss: 0.0114
Epoch 47/100
19/19 0s 11ms/step -
loss: 0.0101 - val_loss: 0.0101

Epoch 48/100
19/19 0s 11ms/step -
loss: 0.0100 - val_loss: 0.0110
Epoch 49/100
19/19 0s 11ms/step -
loss: 0.0098 - val_loss: 0.0097
Epoch 50/100
19/19 0s 11ms/step -
loss: 0.0099 - val_loss: 0.0107
Epoch 51/100
19/19 0s 11ms/step -
loss: 0.0097 - val_loss: 0.0098
Epoch 52/100
19/19 0s 11ms/step -
loss: 0.0096 - val_loss: 0.0105
Epoch 53/100
19/19 0s 11ms/step -
loss: 0.0095 - val_loss: 0.0095
Epoch 54/100
19/19 0s 11ms/step -
loss: 0.0095 - val_loss: 0.0103
Epoch 55/100
19/19 0s 11ms/step -
loss: 0.0094 - val_loss: 0.0095
Epoch 56/100
19/19 0s 11ms/step -
loss: 0.0093 - val_loss: 0.0101
Epoch 57/100
19/19 0s 11ms/step -
loss: 0.0092 - val_loss: 0.0090
Epoch 58/100
19/19 0s 11ms/step -
loss: 0.0091 - val_loss: 0.0101
Epoch 59/100
19/19 0s 11ms/step -
loss: 0.0091 - val_loss: 0.0090
Epoch 60/100
19/19 0s 11ms/step -
loss: 0.0089 - val_loss: 0.0097
Epoch 61/100
19/19 0s 11ms/step -
loss: 0.0089 - val_loss: 0.0087
Epoch 62/100
19/19 0s 11ms/step -
loss: 0.0088 - val_loss: 0.0095
Epoch 63/100
19/19 0s 11ms/step -
loss: 0.0087 - val_loss: 0.0086

```
Epoch 64/100
19/19          0s 11ms/step -
loss: 0.0086 - val_loss: 0.0095
Epoch 65/100
19/19          0s 11ms/step -
loss: 0.0085 - val_loss: 0.0080
Epoch 66/100
19/19          0s 11ms/step -
loss: 0.0085 - val_loss: 0.0091
Epoch 67/100
19/19          0s 11ms/step -
loss: 0.0085 - val_loss: 0.0085
Epoch 68/100
19/19          0s 11ms/step -
loss: 0.0083 - val_loss: 0.0094
Epoch 69/100
19/19          0s 11ms/step -
loss: 0.0083 - val_loss: 0.0082
Epoch 70/100
19/19          0s 11ms/step -
loss: 0.0082 - val_loss: 0.0089
```

```
[16]: plt.plot(history.history["loss"], label="Training Loss")
      plt.plot(history.history["val_loss"], label="Validation Loss")
      plt.xlabel("Epoch")
      plt.ylabel("MAE Reconstruction Loss")
      plt.title("CNN Autoencoder Training Curve")
      plt.legend()
      plt.show()
```



```
[17]: encoded_data = autoencoder.encoder(normal_test_data).numpy()
      decoded_data = autoencoder.decoder(encoded_data).numpy()

      print(encoded_data.shape)
      print(decoded_data.shape)
```

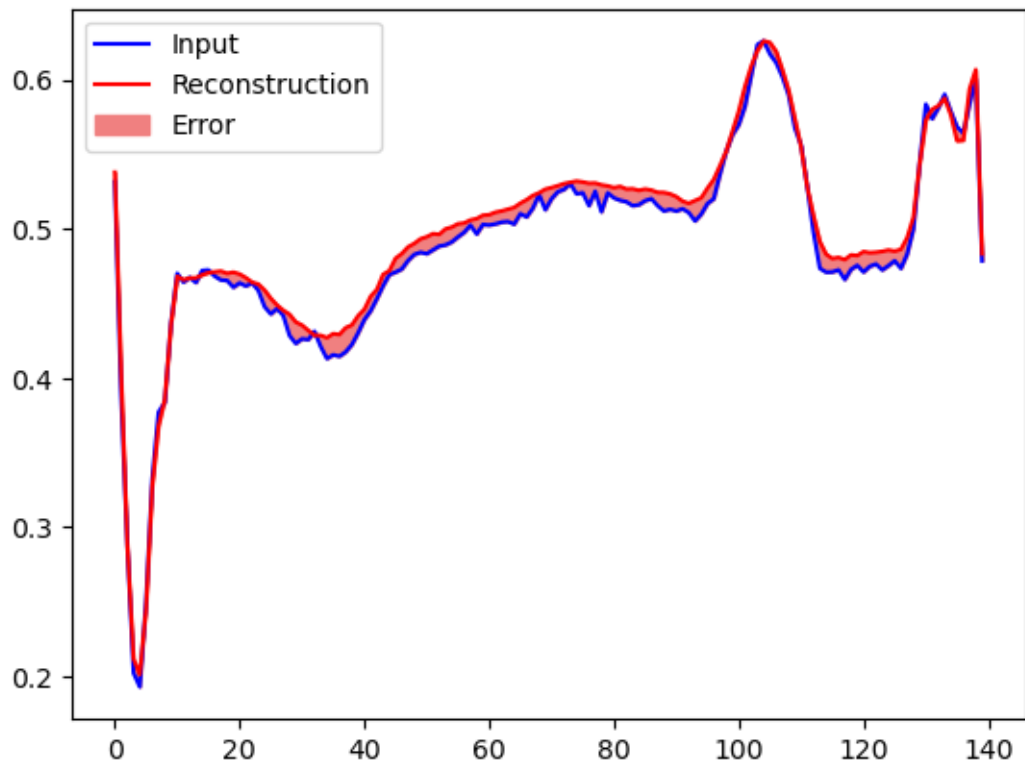
```
(584, 18, 128)
(584, 140, 1)
```

```
[ ]:
```

```
[18]: plt.plot(normal_test_data[0], 'b')
      plt.plot(decoded_data[0].squeeze(), 'r')

      plt.fill_between(
          np.arange(140),
          decoded_data[0].squeeze(),
          normal_test_data[0],
          color='lightcoral'
      )
```

```
plt.legend(labels=["Input", "Reconstruction", "Error"])
plt.show()
```



```
[19]: encoded_data = autoencoder.encoder(anomalous_test_data).numpy()
      decoded_data = autoencoder.decoder(encoded_data).numpy()
```

```
print(encoded_data.shape)
print(decoded_data.shape)
```

```
(416, 18, 128)
```

```
(416, 140, 1)
```

```
[20]: original = anomalous_test_data[0].numpy().squeeze()
      recon = decoded_data[0].squeeze()
```

```
plt.plot(original, 'b')
plt.plot(recon, 'r')
```

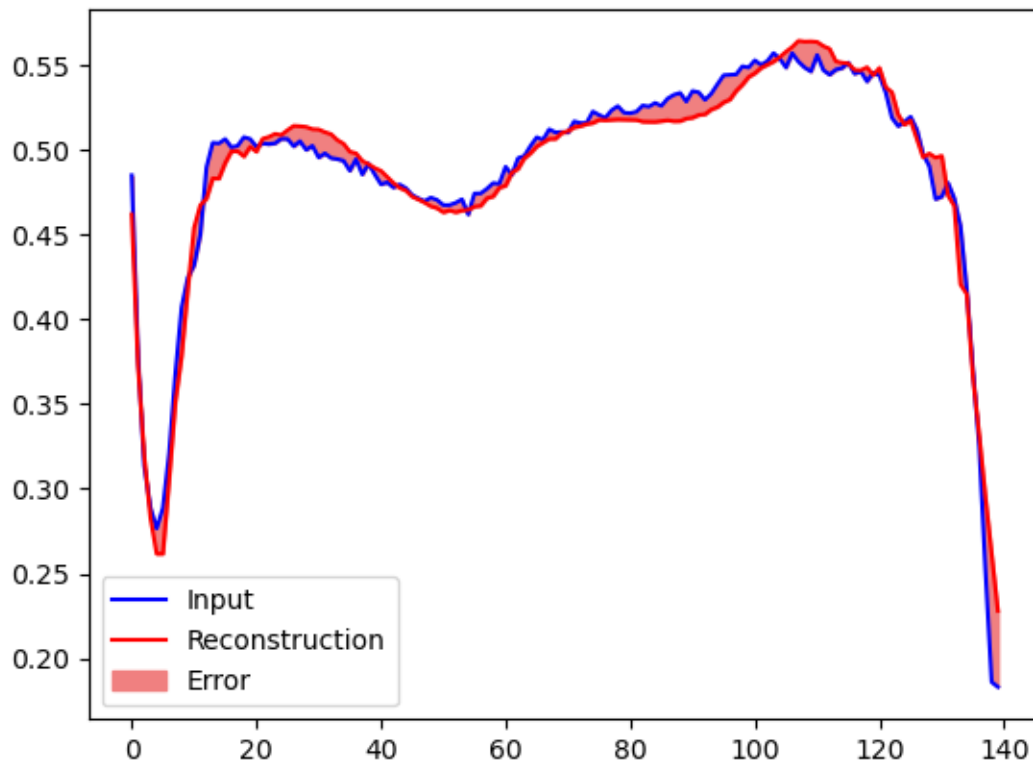
```
plt.fill_between(
    np.arange(140),
```

```

    recon,
    original,
    color='lightcoral'
)

plt.legend(labels=["Input", "Reconstruction", "Error"])
plt.show()

```



[]:

```

[21]: # #Detect anomalies

reconstructions = autoencoder.predict(normal_train_data)

train_loss = tf.reduce_mean(
    tf.abs(reconstructions.squeeze() - normal_train_data),
    axis=1
)

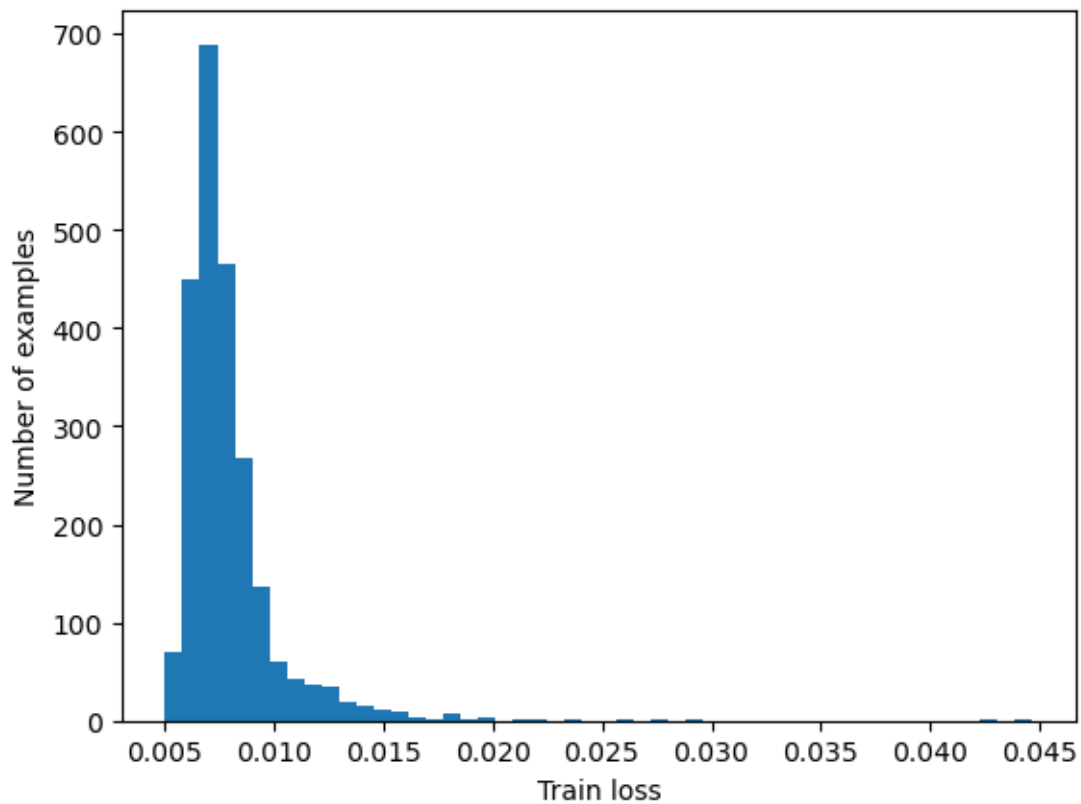
plt.hist(train_loss.numpy(), bins=50)
plt.xlabel("Train loss")
plt.ylabel("Number of examples")

```

```
plt.show()
```

73/73

0s 4ms/step



```
[ ]:
```

```
[22]: #threshold value
threshold = np.mean(train_loss) + np.std(train_loss)
print("Threshold: ", threshold)
```

Threshold: 0.010369964

```
[23]: reconstructions = autoencoder.predict(anomalous_test_data).squeeze()

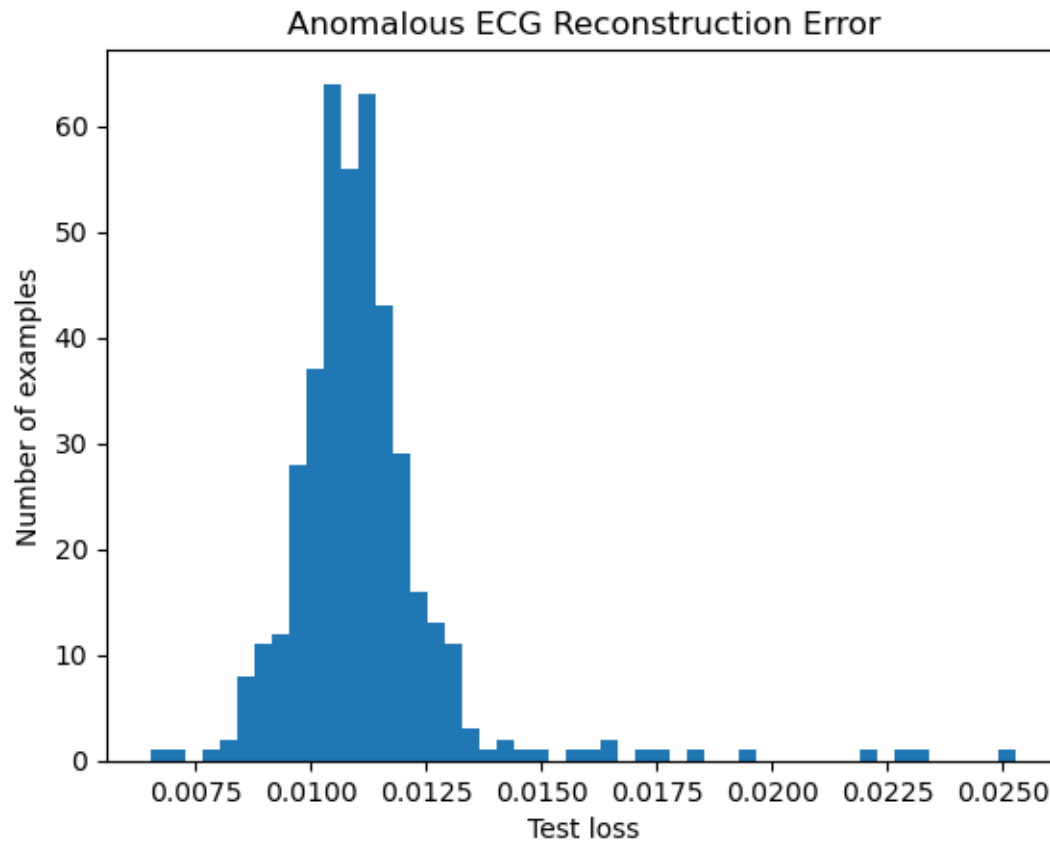
test_loss = tf.reduce_mean(
    tf.abs(reconstructions - anomalous_test_data),
    axis=1
).numpy()

plt.hist(test_loss, bins=50)
plt.xlabel("Test loss")
plt.ylabel("Number of examples")
```

```
plt.title("Anomalous ECG Reconstruction Error")
plt.show()
```

13/13

0s 3ms/step



```
[24]: def predict(model, data, threshold):
    reconstructions = model(data)

    # Ensure both tensors have the same shape
    reconstructions = tf.squeeze(reconstructions)
    data = tf.squeeze(data)

    loss = tf.reduce_mean(
        tf.abs(reconstructions - data),
        axis=1
    )

    return (loss > threshold).numpy().astype(int), loss.numpy()

def print_stats(predictions, labels):
    print("Accuracy = {}".format(accuracy_score(labels, predictions)))
```



```
print("Precision = {}".format(precision_score(labels, predictions)))
print("Recall = {}".format(recall_score(labels, predictions)))
```

```
[25]: # Predict on test data
preds, scores = predict(autoencoder, test_data, threshold)

# Convert labels to
test_labels = test_labels.astype(int)

# Print metrics
print_stats(preds, test_labels)
```

```
Accuracy = 0.836
Precision = 0.8579545454545454
Recall = 0.7259615384615384
```

```
[26]: ## Result

from sklearn.metrics import confusion_matrix

# Ensure labels are integers
test_labels = test_labels.astype(int)

# Compute confusion matrix
cm = confusion_matrix(test_labels, preds)

print("Confusion Matrix:")
print(cm)
```

```
Confusion Matrix:
[[534  50]
 [114 302]]
```

```
[33]: from sklearn.metrics import confusion_matrix

# Ensure labels are integers
test_labels = test_labels.astype(int)

# Compute confusion matrix
cm = confusion_matrix(test_labels, preds)

print("Confusion Matrix:")
print(cm)

# Accuracy
accuracy = (cm[0,0] + cm[1,1]) / cm.sum() * 100
```

```

# Percentages
cm_percent = cm / cm.sum() * 100

# Build annotation labels (same shape as cm)
labels = np.empty_like(cm).astype(str)
for i in range(cm.shape[0]):
    for j in range(cm.shape[1]):
        labels[i, j] = f"{cm[i, j]}\n({cm_percent[i, j]:.1f}%)"

# Plot
plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=labels, fmt="", cmap="Blues",
            xticklabels=["Normal", "Anomaly"],
            yticklabels=["Normal", "Anomaly"])

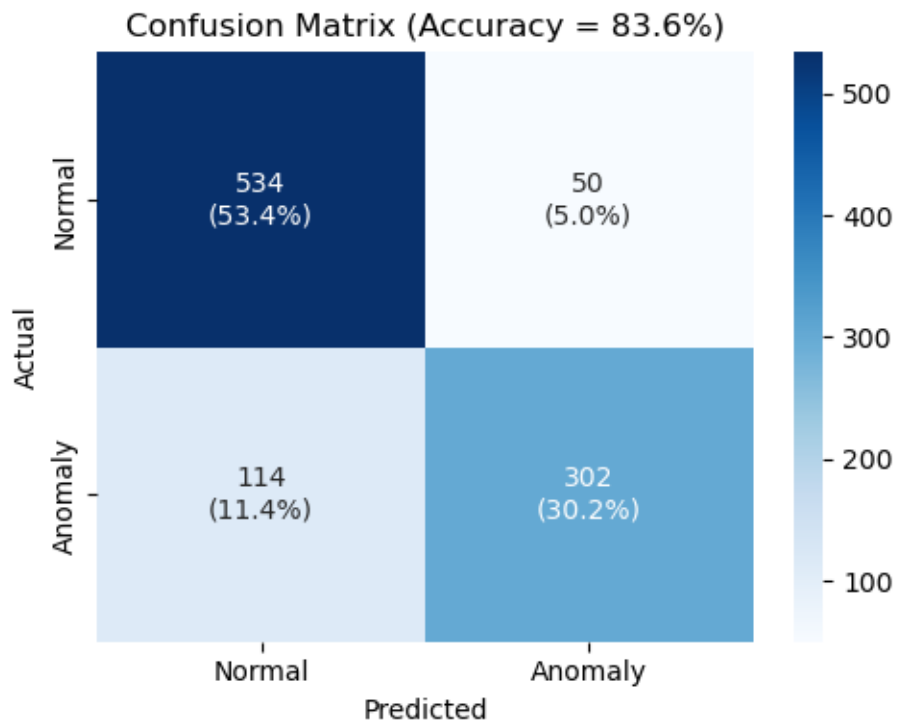
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Confusion Matrix (Accuracy = {accuracy:.1f}%)"
plt.tight_layout()
plt.show()

```

Confusion Matrix:

```
[[534  50]
```

```
 [114 302]]
```



```
[40]: ## >> Test prediction by selecting one idx #
```

```
idx = 998 # choose a sample index 1 to 999
```

```
plt.figure(figsize=(12, 4))
```

```
plt.plot(test_data[idx], label="ECG Signal")
```

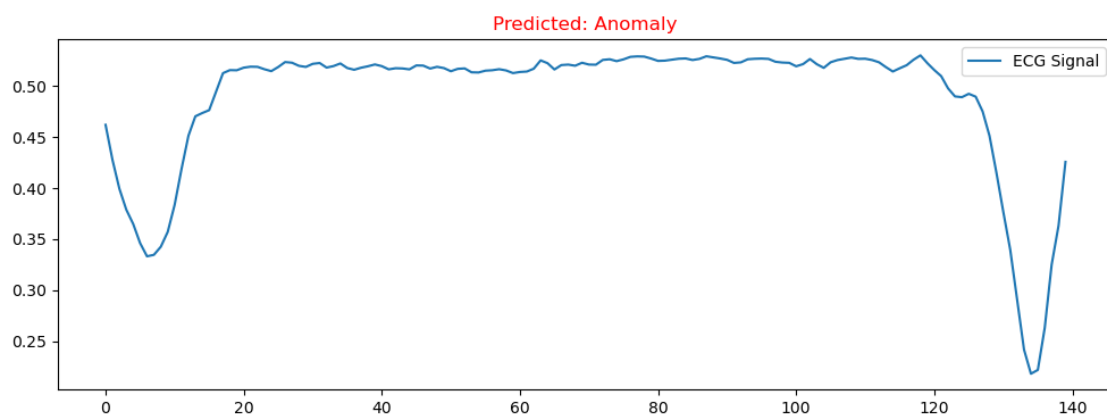
```
label = preds[idx]
```

```
color = "red" if label == 1 else "green"
```

```
plt.title(f"Predicted: {'Anomaly' if label==1 else 'Normal'}", color=color)
```

```
plt.legend()
```

```
plt.show()
```



```
[ ]:
```